

SemanticSplat: Graph-Pruned Semantic Search over Captured 3D Gaussian Splatting Scenes

Mahmoud Mousatat, Leo Vizan, Nikita Shankin, Telman Nuruzov, Alexandr Medvedev

Abstract

3D Gaussian Splatting (3DGS) provides photorealistic rendering but no semantic structure for natural-language environment queries such as “find a place where I can sit” or “where is the screen in the conference room?” We present SemanticSplat, a rendering-first pipeline that builds a hierarchical semantic index from captured views and answers queries by traversing this index top-down, pruning irrelevant regions before leaf-level confirmation. To isolate the effect of hierarchy, we compare graph-pruned search against a flat baseline that scans the *same* semantic items under an identical protocol. On five manually captured indoor and outdoor scenes (96 views, 1,046 semantic items) and 150 benchmark queries spanning six query types, graph traversal reduces average views checked by 71.4% and input tokens by 65.1% relative to flat search, while retrieval quality stays comparable: hit@1 is 0.792 (graph) vs. 0.768 (flat) and hit@3 is 0.904 vs. 0.928 on the 125 queries with verified view labels. A per-query-type analysis shows savings ranging from 53% on multi-hop queries to 90% on negative queries, revealing where hierarchy helps most. We report the setting honestly: manual reference labels rather than independent ground truth, and lexical stub matching rather than live provider-backed VLM evaluation.

1 Introduction

3D Gaussian Splatting [10] enables real-time, photorealistic novel-view synthesis, but the representation encodes only geometry and appearance. It does not know that a cluster of Gaussians is a “chair,” a “reception desk,” or a “ballroom stage.” For downstream tasks in robotics, AR/VR, and natural-language interaction, a scene must be *queryable*: a user should be able to ask for an object, an attribute, or an affordance and receive the relevant viewpoint.

A natural approach is exhaustive (*flat*) search: describe every captured view to a language model and let it pick the best match. This is simple but scales poorly—context length and cost grow linearly with the number of views, and unrelated rooms constantly compete for attention. Existing semantic-3D methods take two other routes. Language-field methods distill vision-language features into radiance fields or Gaussians [11, 15], which support point-wise relevance but lack explicit structure for context control. Explicit 3D scene-graph methods lift 2D detections into object graphs [5, 13], which can be fragile for open-vocabulary hierarchy construction.

SemanticSplat takes a rendering-first, hierarchy-first approach. We capture posed RGB views from a 3DGS scene, annotate them with structured records (objects, landmarks, room cues, and a short summary), organize them into a zone–region–object tree, and answer queries by traversing that tree top-down. Our central research question is deliberately narrow and testable: *when a graph method and a flat method operate on identical semantic items, does hierarchical pruning reduce search cost without sacrificing useful retrieval quality?*

Contributions.

- A rendering-first semantic-map layer over captured 3DGS views, with MCP-backed infrastructure and query-flow logging.
- A controlled same-input benchmark comparing graph traversal to flat search over five captured scenes and 150 queries, measuring views checked, input tokens, and GT-aware hit@k.
- A per-scene and per-query-type analysis that quantifies *where* hierarchy helps, plus an affordance-expansion layer for intent-style queries.
- Honest scoping: manual reference labels, stub evaluation mode, and baseline smokes reported without superiority claims.

2 Related Work

Language fields and 3DGS semantics. LERF [11], LangSplat [15], Semantic Gaussians [6], and LEGS [17] embed or distill vision-language features into radiance fields or Gaussians for open-vocabulary pointing and robot-scale mapping. Their native outputs are relevancy maps or semantic Gaussians rather than explicit traversal traces; they typically lack a zone–region–object hierarchy for context pruning.

Open-vocabulary 3D scene graphs. ConceptGraphs [5], OpenScene [14], ConceptFusion [8], and BBQ [13] build explicit graphs from lifted 2D detections, point clouds, or RGB-D association. These methods target object graphs, relational grounding, and planning; we instead evaluate whether a curated hierarchical index over captured views reduces same-input search cost for environment-understanding queries.

Hierarchical scene graphs and benchmarks. HOV-SG [16] and Hydra [7] construct multi-level spatial graphs with stronger geometric priors for navigation and robotics. OpenLex3D [9] and ScanRefer [1] define tiered open-vocabulary and referring-expression benchmarks with 3D IoU and localization metrics on RGB-D scans. Our captured scenes lack independent object boxes or masks at that scale, so we report internal efficiency and GT-aware hit@k on verified view labels instead.

Token and context efficiency. Prompt compression [12], AttentionRAG [3], Provence [2], TeaRAG [18], and S-PathRAG [4] study pruning retrieved context and graph-derived evidence for LLMs. Token count is not a standard 3D scene-understanding metric; we draw on this line to frame views checked and input tokens as internal graph-vs-flat efficiency measures, not as claims of superiority over RAG systems.

Positioning. We do not claim superiority over Concept-Graphs, LangSplat, or external graph builders. Our measured comparison is strictly internal: flat lexical search vs. graph traversal on identical scenes, semantic items, and query strings.

3 Method

3.1 Capture and semantic index

A user navigates a 3DGS scene in the browser and captures posed views (RGB, depth, and camera pose). Each view is annotated with a structured record listing visible objects, landmarks, room cues, and a short (10–15 word) natural-language summary. These records form the *semantic index*: the atomic units that both the graph and flat methods search. Keeping the summaries short bounds the per-view token cost during traversal.

3.2 Hierarchical tree

Semantic items are organized into a three-level tree: *zones* (e.g. lobby, ballroom), *regions* within a zone, and *object/view leaves*. Each internal node stores an aggregated summary of its subtree, so a traversal can decide whether to descend into or prune a branch by reading a compact node summary rather than every underlying view.

3.3 Query traversal and flat baseline

A query is decomposed into structured constraints (target object, attributes, spatial relations, query type). The tree is traversed top-down: at each node the model compares the query against the node summary and prunes irrelevant zones and regions before confirming at the leaf level. The *flat* baseline removes the hierarchy entirely and scans the same semantic items exhaustively. Because both methods consume identical

Scene	Views	Items	Tree nodes
Conference Hall	20	207	200
Museum	20	225	221
Theater	20	226	220
Outdoor (drone)	16	169	158
Outdoor (street)	20	219	216
Total	96	1,046	1,015

Table 1: Five-scene dataset composition. Semantic items are the shared search units for both the graph and flat methods.

inputs, any difference in cost or quality is attributable to the pruning structure, not to different data.

3.4 Affordance-aware expansion

Intent queries are mapped to affordance categories before ranking—for example *sittable* → chair, sofa, bench; *exit* → door, stairs, sign. This deterministic expansion lets natural, goal-oriented phrasings resolve to concrete object candidates without a learned model.

3.5 Evaluation protocol

Graph and flat methods share scenes, semantic items, query strings, and benchmark labels. Efficiency metrics are views checked, input tokens, context size, and infrastructure elapsed time. Quality metrics (hit@1, hit@3) are computed only for queries with verified expected view IDs. We treat manual annotations as *reference labels*, not independent ground truth, and all reported five-scene runs use lexical stub matching rather than live VLM API calls.

4 Dataset and Benchmark

The evaluation uses five captured scenes—three indoor (Conference Hall, Museum, Theater) and two outdoor (drone, street). Table 1 summarizes their composition: 96 annotated views and 1,046 semantic items organized into deterministic trees. The benchmark contains 150 queries evenly split across six query types (simple object, attribute, relational, functional, multi-hop, negative), 25 per type. Of these, 125 carry verified expected view IDs used as the quality denominator; the remaining 25 (mostly negatives with no target view) are excluded from hit@k.

5 Experiments

5.1 Main result: graph vs. flat

Table 2 and Figure 1 report averages over all 150 queries. Graph traversal checks 5.39 views on average versus 19.2 for flat search (71.4% fewer) and consumes 1,061.6 input tokens versus 3,104 (65.1% fewer). Retrieval quality is preserved:

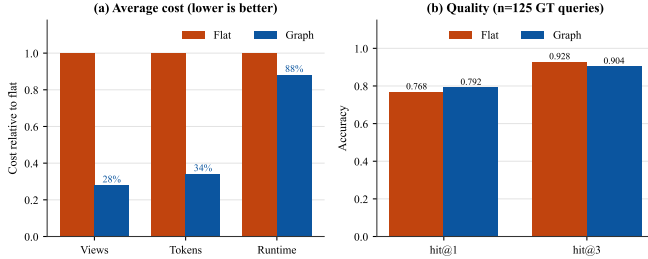


Figure 1: Five-scene same-input comparison (150 queries). Left: average views, tokens, and infrastructure runtime. Right: hit@1 and hit@3 on the 125 queries with verified view labels.

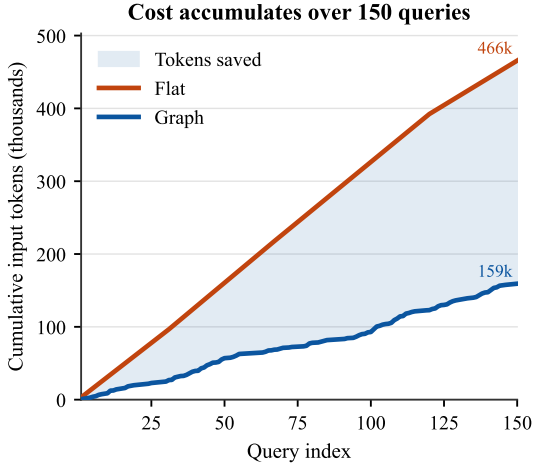


Figure 2: Cumulative input tokens over the 150-query benchmark. The flat baseline grows almost linearly, while graph traversal stays far below; the shaded region is the accumulated saving (466k vs. 159k tokens overall).

graph hit@1 is slightly higher (0.792 vs. 0.768) while flat retains a small hit@3 edge (0.928 vs. 0.904), consistent with hierarchy trading a little recall breadth for large cost reductions. Figure 2 shows the effect accumulating across the benchmark: over 150 queries flat search consumes 466k input tokens versus 159k for graph traversal.

5.2 Where does hierarchy help? Per-type analysis

Aggregate numbers hide substantial variation. Figure 3 and Table 3 break token savings down by query type. Negative queries (“is there an X?” when no X exists) benefit most: the tree prunes entire zones early, yielding 89.5% savings and only 1.6 views checked. Simple-object, attribute, and functional queries land in a 63–67% band. Multi-hop queries—which reference several entities across regions—show the smallest savings (53.0%) and the highest graph view count (7.4), because they force the traversal to descend into multiple branches. This is the expected failure mode of hierarchy: it prunes best when a query localizes to one part of the tree.

Metric	Flat	Graph
Views checked	19.2	5.39
Input tokens	3,104.0	1,061.6
Elapsed (ms, infra)	14.1	12.5
hit@1 (125 GT queries)	0.768	0.792
hit@3 (125 GT queries)	0.928	0.904

Table 2: Same-input graph vs. flat on five captured scenes: 71.4% fewer views and 65.1% fewer tokens at comparable quality. The quality denominator excludes 25 queries without verified view labels.

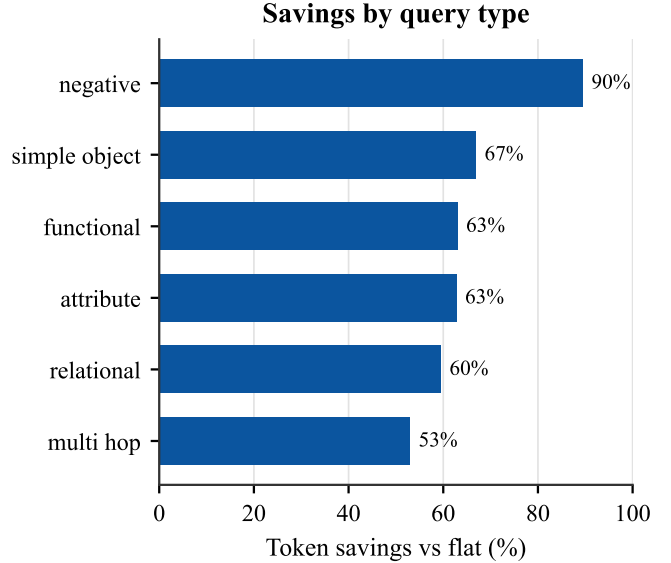


Figure 3: Graph token savings relative to flat, by query type (25 queries each). Hierarchy helps most when a query localizes to a single zone (negatives, simple objects) and least when it spans regions (multi-hop).

5.3 Per-scene consistency

Figure 4 shows that the efficiency gain holds across all five scenes for both views and tokens, though its magnitude tracks scene structure. The Theater, with a clean zone layout, yields the largest token savings (80.9%), while the outdoor drone scene—fewer views and a flatter tree—yields the smallest (50.4%). No scene regresses, indicating the benefit is a property of the hierarchy rather than of one favorable environment.

5.4 Legacy demo scene

On an earlier single-scene Conference Hall demo (19 views, a 15-node tree) with 12 queries, graph traversal reduces input tokens by $4.1\times$ and views checked by $4.8\times$, with an estimated $4.0\times$ latency speedup under a documented LLM timing model. We keep this result separate from the five-scene averages because it uses an estimated timing model rather than the stub measurements above.

Query type	Graph views	Graph tokens	Savings
Negative	1.6	325	89.5%
Simple object	5.0	1,027	66.9%
Functional	5.9	1,148	63.0%
Attribute	6.2	1,154	62.8%
Relational	6.2	1,257	59.5%
Multi-hop	7.4	1,459	53.0%

Table 3: Per-query-type breakdown (flat baseline: 19.2 views, 3,104 tokens). Savings decrease as queries span more of the tree.

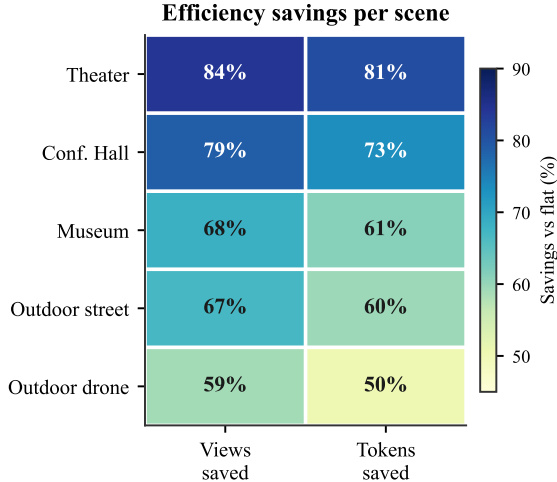


Figure 4: Per-scene efficiency savings (view and token reduction vs. flat, darker = larger saving). Graph traversal helps in every scene; the gain is largest where zone structure is clearest (Theater) and smallest for the flatter outdoor drone scene.

5.5 Case studies and failure analysis

To probe behavior beyond aggregate numbers, we run five curated room-scoped queries on the demo scene and classify the returned view by room (Figure 5). These cases surface three qualitatively different regimes.

Efficiency at equal correctness. On the three “screen” queries, graph traversal and flat search return the *same* correct view, but graph reaches it after checking 2 views ($\approx 2.9k$ tokens) versus flat’s full 19-view scan ($\approx 23k$ tokens)—an order-of-magnitude cost reduction with no loss of correctness. This is the intended operating regime: when a query localizes to one zone, the tree confirms early.

Failure mode 1: uneven pruning. For “find the projector in the ballroom,” the target view sits in a large, densely populated zone. Graph traversal still reaches the correct room, but only after checking 16 of 19 views, so its cost approaches the flat baseline. Hierarchy provides little benefit when the answer is spread across a broad zone rather than isolated in a narrow branch.

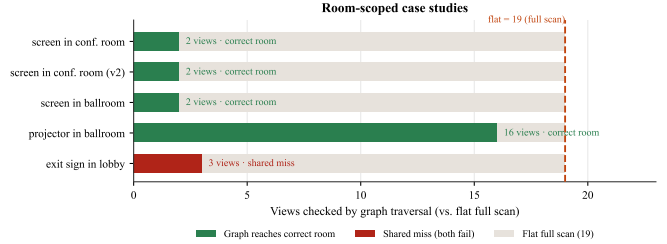


Figure 5: Room-scoped case studies on the demo scene. Each colored bar is the number of views graph traversal checks; the faint bar and dashed line mark flat’s full 19-view scan. Graph reaches the same correct room far cheaper on isolated targets (2 views), approaches the baseline on the spread-out projector query (16 views), and fails cheaply on the shared miss (3 views).

Failure mode 2: shared retrieval miss. For “where is the exit sign in the lobby,” both methods miss: the current lexical stub index does not surface a matching view. Graph traversal nevertheless fails *cheaply*, checking 3 views before returning empty, whereas flat search still pays for a full scan. This isolates a limitation of the retrieval backend (not the hierarchy) and shows that pruning also bounds the cost of failures.

We note that, on the current scene state, flat search does *not* exhibit wrong-room errors on these queries; both methods agree on the room when they succeed. The value of hierarchy here is cost control at matched quality, not higher accuracy.

5.6 Baselines and scope

ConceptGraphs and LangSplat smoke tests confirm adapter feasibility but do not provide five-scene accuracy or efficiency comparisons, so we make no head-to-head claim. Live and cached-live provider evaluation, and ScanNet-scale benchmarking, are out of scope for this study.

6 Discussion

Three patterns emerge. First, the savings are driven by *early pruning*: whenever a query can be localized to a zone from its summary, the tree discards the majority of views before any leaf check, which is why negatives and simple objects gain most. Second, the small hit@3 gap in favor of flat search reflects a genuine trade-off—exhaustive search never prunes a correct-but-distant candidate, while the tree occasionally does. In cost terms, however, the graph reaches comparable hit@1 while reading roughly one third of the tokens. Third, because both methods share inputs, these effects are attributable to structure alone, which is the property we set out to isolate. Together this suggests hierarchy is most valuable as a *cost-control* mechanism for environment queries, with quality parity rather than a quality win.

7 Limitations

- **Manual reference labels:** expected objects and views are verified from annotations, not independent annotators, so absolute hit@k should be read as relative graph-vs-flat evidence.
- **Stub evaluation mode:** five-scene runs use lexical stub matching, not live VLM calls; live behavior may differ.
- **No metric 3D localization:** independent 3D boxes are unavailable, so 3D IoU is not reported.
- **Uneven benefit:** multi-hop queries retain high view counts; savings depend on query type and tree shape.
- **Baselines:** external methods are literature context plus minimal smokes, not same-input comparisons.

8 Conclusion

SemanticSplat shows that hierarchical graph pruning substantially reduces the cost of semantic search over captured 3DGS scenes while preserving retrieval quality, under a controlled protocol where graph and flat methods consume identical semantic items. Across five scenes, six query types, and 150 queries, graph traversal cuts views checked by 71.4% and tokens by 65.1% at comparable hit@1, with the largest gains on queries that localize to a single zone. Future work includes live VLM evaluation, richer zone–region–object graphs, independent ground truth, and fair same-input comparison against external scene-graph systems.

References

- [1] Dave Zhenyu Chen, Angel X. Chang, and Matthias Nießner. Scanrefer: 3d object localization in rgb-d scans using natural language. In *European Conference on Computer Vision*, 2020.
- [2] Nadezhda Chirkova, Thibault Formal, Vassilina Nikoulina, and Stéphane Clinchant. Provence: efficient and robust context pruning for retrieval-augmented generation. *arXiv preprint arXiv:2501.16214*, 2025.
- [3] Yixiong Fang, Tianran Sun, Yuling Shi, and Xiaodong Gu. Attentionrag: Attention-guided context pruning in retrieval-augmented generation. *arXiv preprint arXiv:2503.10720*, 2025.
- [4] Rong Fu, Yemin Wang, Tianxiang Xu, Yongtai Liu, Weizhi Tang, Wangyu Wu, Xiaowen Ma, and Simon Fong. S-pathrag: Semantic-aware shortest-path retrieval augmented generation for multi-hop knowledge graph question answering. *arXiv preprint arXiv:2603.23512*, 2026.
- [5] Qiao Gu, Alihusein Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning. *arXiv preprint arXiv:2309.16650*, 2023.
- [6] Jun Guo, Xiaojian Ma, Yue Fan, Huaping Liu, and Qing Li. Semantic gaussians: Open-vocabulary scene understanding with 3d gaussian splatting. *arXiv preprint arXiv:2403.15624*, 2024.
- [7] Nathan Hughes, Yun Chang, and Luca Carlone. Hydra: A real-time spatial perception system for 3d scene graph construction and optimization. *arXiv preprint arXiv:2201.13360*, 2022.
- [8] Krishna Murthy Jatavallabhula, Alihusein Kuwajerwala, Qiao Gu, Mu Omama, Tao Chen, Alaa Maalouf Li, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. Conceptfusion: Open-set multimodal 3d mapping. *arXiv preprint arXiv:2302.07241*, 2023.
- [9] Christina Kassab, Sacha Morin, Martin Buchner, Matias Matamala, Kumaraditya Gupta, Abhinav Valada, Liam Paull, and Maurice Fallon. Openlex3d: A tiered evaluation benchmark for open-vocabulary 3d scene representations. *arXiv preprint arXiv:2503.19764*, 2025.
- [10] Bernhard Kerbl, Georgios Kopanas, Laura Leal-Taixé, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 2023.
- [11] Justin Kerr, Chung Min Kim, Ken Goldberg, Angjoo Kanazawa, and Matthew Tancik. Lerf: Language embedded radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [12] Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. Prompt compression for large language models: A survey. *arXiv preprint arXiv:2410.12388*, 2024.
- [13] Sergey Linok, Tatiana Zemskova, Svetlana Ladanova, Roman Titkov, Dmitry Yudin, Maxim Monastyrny, and Aleksei Valenkov. Beyond bare queries: Open-vocabulary object grounding with 3d scene graph. *arXiv preprint arXiv:2406.07113*, 2024.
- [14] Songyou Peng, Kyle Genova, Chiyu Jiang, Andrea Tagliasacchi, Marc Pollefeys, and Thomas Funkhouser. Openscene: 3d scene understanding with open vocabularies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.
- [15] Minghan Qin, Wanhua Li, Jiawei Zhou, Haoqian Wang, and Hanspeter Pfister. Langsplat: 3d language gaussian splatting. *arXiv preprint arXiv:2312.16084*, 2024.
- [16] Abdelrhman Werby, Chenguang Huang, Martin Buchner, Abhinav Valada, and Wolfram Burgard. Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation. In *Robotics: Science and Systems*, 2024.
- [17] Justin Yu, Kush Hari, Kishore Srinivas, Karim El-Refai, Adam Rashid, Chung Min Kim, Justin Kerr, Richard Cheng, Muhammad Zubair Irshad, Ashwin Balakrishna, Thomas Kollar, and Ken Goldberg. Language-embedded gaussian splats (legs): Incrementally building room-scale representations with a mobile robot. *arXiv preprint arXiv:2409.18108*, 2024.
- [18] Chao Zhang, Yuhao Wang, Derong Xu, Haoxin Zhang, Yuanjie Lyu, Yuhao Chen, Shuochen Liu, Tong Xu, Xiangyu Zhao, Yan Gao, Yao Hu, and Enhong Chen. Tearag: A token-efficient agentic retrieval-augmented generation framework. *arXiv preprint arXiv:2511.05385*, 2025.